Commitment Schemes

November 9, 2026

Blockstream Research

research.blockstream.com

github.com/BlockstreamResearch/zk-book



Plan

1 Commitments Overview

2 Hash-based Commitments

3 Pedersen Commitments

4 Polynomial Commitments

5 KZG Commitments

Commitments Overview

Commitment Definition

Definition

A cryptographic commitment scheme allows one party to commit to a chosen statement (such as a value, vector, or polynomial) without revealing the statement itself. The commitment can be revealed in full or in part at a later time, ensuring the integrity and secrecy of the original statement until the moment of disclosure.

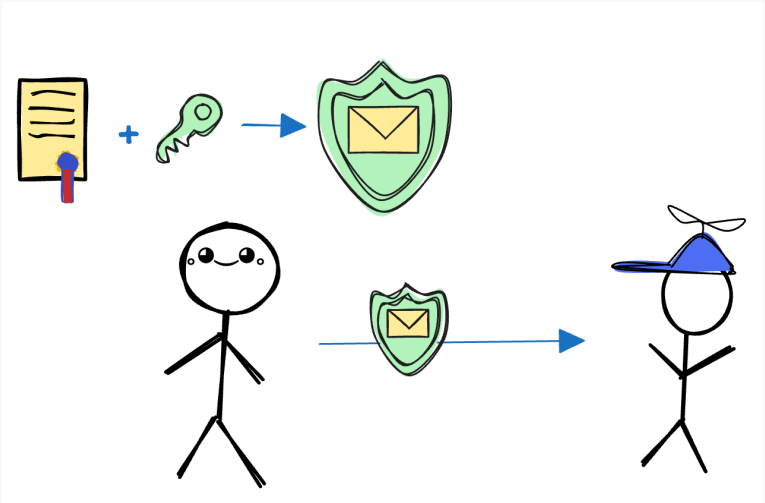


Figure: A letter in a locked box: the friend cannot read it, and you cannot change it.

Commitment Scheme

Definition (Commitment Scheme)

A commitment scheme Π is a tuple of two algorithms $(\text{Setup}, \text{Com})$:

1. $\text{Setup}(1^\lambda) \to \text{pp}$: returns public parameters pp for both the committer (prover) and the verifier;
2. $\text{Com}(\text{pp}, m) \to c$: returns a commitment c to the message m using the public parameters pp.

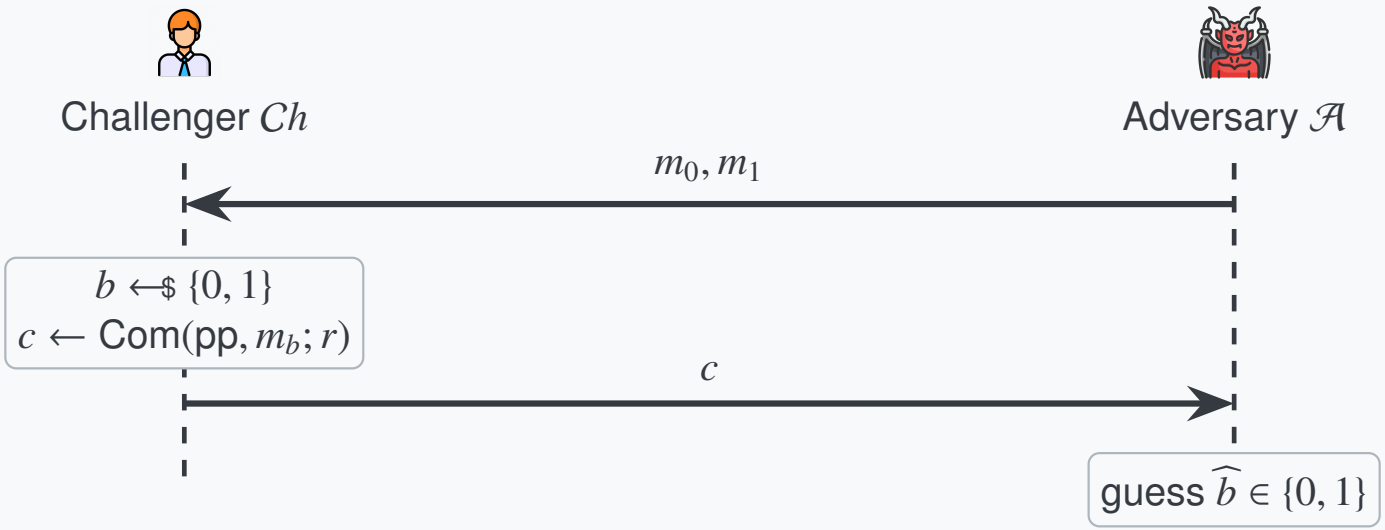
To open c, the committer simply reveals m, and the verifier checks whether $\text{Com}(\text{pp}, m) = c$.

Remark

In many applications, the commitment also takes a random component r together with the message, which we write as $\text{Com}(\text{pp}, m; r)$. Such a component is necessary for the **hiding** property.

Hiding

Informally: the verifier learns nothing about m from the commitment c.
Both parties start with $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$.



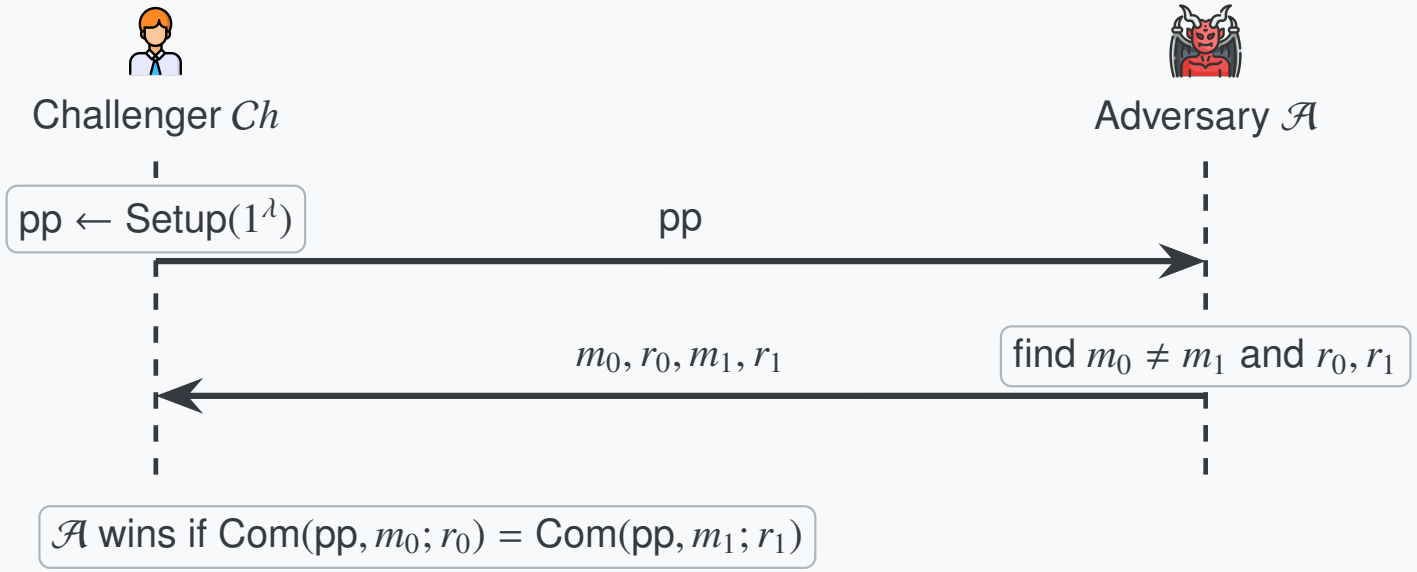
Definition

Π is **hiding** if for any efficient adversary $\mathcal{A}$:

$$\left| \Pr\left[b = \widehat{b} \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda),\ m_0, m_1 \leftarrow \mathcal{A}(\mathsf{pp}) \\ b \leftarrow\$ \{0,1\},\ c \leftarrow \mathsf{Com}(\mathsf{pp}, m_b; r) \\ \widehat{b} \leftarrow \mathcal{A}(c) \end{array} \right] - \frac{1}{2} \right| = \mathsf{negl}(\lambda).$$

Binding

Informally: the committer cannot open the same commitment c to two different messages.



Definition

Π is **binding** if for any efficient adversary $\mathcal{A}$:

$$\Pr\left[\begin{array}{l|l} \text{Com}(\text{pp}, m_0; r_0) = \text{Com}(\text{pp}, m_1; r_1) & \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ \text{and } m_0 \neq m_1 & (m_0, m_1, r_0, r_1) \leftarrow \mathcal{A}(\text{pp}) \end{array}\right] = \text{negl}(\lambda).$$

Perfect vs Computational Security

Definition

The scheme is **perfectly** hiding (binding) if the corresponding advantage is *exactly zero* for any, not necessarily efficient, adversary. Otherwise, if the advantage is only negligible for efficient adversaries, the scheme is **computationally** hiding (binding).

Remark

No commitment scheme is perfectly hiding and perfectly binding at the same time.

Why? Suppose Π is perfectly hiding and take $m_0 \neq m_1$. Then $\mathsf{Com}(\mathsf{pp}, m_0; r)$ and $\mathsf{Com}(\mathsf{pp}, m_1; r)$ have *identical* distributions over random r. So for any r_0 there exists r_1 with

$$\mathsf{Com}(\mathsf{pp}, m_0; r_0) = \mathsf{Com}(\mathsf{pp}, m_1; r_1).$$

An unbounded adversary finds such r_0, r_1 and wins the binding game with probability 1.

Hash-based Commitments

Hash-based Commitments

The simplest way to build a commitment scheme is to use a cryptographic hash function H.

Definition

The **hash-based commitment** Π_{H} is defined as follows:

1. $\mathsf{Setup}(1^\lambda)$ initializes the hash function H, fixes the message length ℓ and the blinding factor length k, and outputs $\mathsf{pp} \leftarrow (\mathsf{H}, \ell, k)$.
2. $\mathsf{Com}(\mathsf{pp}, m; r) \to c$, for a message $m \in \{0,1\}^\ell$, samples the blinding factor $r \leftarrow\!\!\$\ \{0,1\}^k$ and outputs

$$c \leftarrow \mathsf{H}(m \,\|\, r).$$

To open c, the committer reveals (m, r), and the verifier recomputes $\mathsf{H}(m \,\|\, r)$.

Hash-based Commitments: Security

Lemma

Π_{H} is hiding in the random oracle model for H.

In the random oracle model, $\mathsf{H}(m \,\|\, r)$ for a fresh random r is a uniformly random string, independent of m.

Lemma

If H is collision-resistant, then Π_{H} is computationally binding.

Proof. Suppose an efficient $\mathcal{A}$ breaks binding: it outputs $m_0 \neq m_1$ and r_0, r_1 with $\mathsf{H}(m_0 \,\|\, r_0) = \mathsf{H}(m_1 \,\|\, r_1)$. Since messages and blinding factors have fixed lengths, $m_0 \,\|\, r_0 \neq m_1 \,\|\, r_1$, so this is a collision of H, which contradicts its collision resistance.

Pedersen Commitments

Pedersen Commitment

Pedersen commitments allow us to represent arbitrarily large vectors with a single elliptic curve point, while hiding any information about the vector.

We use a public group $\mathbb{G}$ of order q (written multiplicatively) and two random generators g and h with $h = g^{\rho}$. The secret ρ must be unknown to anyone, otherwise **binding** is violated.

💬 *Transparent random points generation*

Pick a publicly chosen seed (like a hash of the project name) and hash it to obtain a coordinate x. If x does not yield a point on the curve, increment it until a valid point (x, y) is found. Since the points come from a hash of a fixed public seed, no one knows the discrete logarithms between them.

Pedersen Commitment

Definition

The **Pedersen commitment scheme** Π_{Ped} is defined as follows:

- Setup$(1^\lambda) \to$ pp: the prover and the verifier agree on points g, h in the elliptic curve group $\mathbb{G}$ of order $q := \#\mathbb{G} = \Theta(2^\lambda)$.
- Com(pp, $m \in \mathbb{Z}_q; r) \to c \in \mathbb{G}$: the prover selects a blinding factor $r \leftarrow\!\!\$\ \mathbb{Z}_q$ and outputs

$$c \leftarrow g^m h^r.$$

To open c, the prover reveals (m, r), and the verifier checks that $g^m h^r = c$.

Pedersen Commitment: Trapdoor

If the discrete logarithm ρ of h is leaked, the **prover** can break binding:

$$c = g^m h^r = g^m g^{r\rho} = g^{m+r\rho}.$$

For example, $(m + \rho, r - 1)$ is another valid opening of the same commitment:

$$g^{m+\rho} h^{r-1} = g^{m+\rho+(r-1)\rho} = g^{m+r\rho} = c.$$

Takeaway

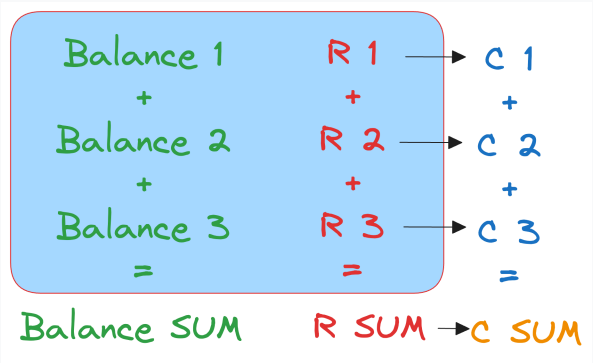
Whoever knows $\rho = \log_g h$ can open c to *any* message. That is why g and h must be generated transparently.

Pedersen Commitment: Homomorphism

Unlike hash-based commitments, Pedersen commitments are **homomorphic**, so they can be accumulated. Consider two pairs $(m_0, r_0), (m_1, r_1)$:

$$c_0 \leftarrow g^{m_0} h^{r_0}, \quad c_1 \leftarrow g^{m_1} h^{r_1},$$
$$c_0 c_1 = g^{m_0+m_1} h^{r_0+r_1} = \mathsf{Com}(\mathsf{pp}, m_0 + m_1; r_0 + r_1).$$

Example: in confidential transactions, one checks that the inputs and outputs balance, $x_0 + \cdots + x_n = z_0 + \cdots + z_m$, without revealing individual amounts.



Pedersen Commitment: Security

Lemma

Π_{Ped} is *perfectly* hiding and *computationally* binding under the discrete logarithm assumption.

Perfect hiding. The map $\mathbb{Z}_q \to \mathbb{G}$, $r \mapsto h^r$ is a bijection, so h^r is uniformly distributed in $\mathbb{G}$. Hence, for any fixed $m \in \mathbb{Z}_q$, the commitment $g^m h^r$ is also uniformly distributed in $\mathbb{G}$, independently of m.

Note

Even an unbounded adversary learns nothing: every $c \in \mathbb{G}$ can be opened to every m with some r. This matches the impossibility result: binding can only be computational.

Pedersen Commitment: Security

Computational binding. Suppose an efficient adversary $\mathcal{A}$ outputs two openings $(m_0, r_0) \neq (m_1, r_1)$ of the same commitment:

$$g^{m_0} h^{r_0} = g^{m_1} h^{r_1}.$$

Rearranging yields

$$h^{r_1 - r_0} = g^{m_0 - m_1}.$$

Since the openings differ, $r_0 \neq r_1$ (otherwise $m_0 = m_1$ as well). Therefore, $\mathcal{A}$ finds the discrete logarithm of h:

$$\rho = \log_g h = (m_0 - m_1)(r_1 - r_0)^{-1} \in \mathbb{Z}_q,$$

which contradicts the discrete logarithm assumption.

Pedersen Vector Commitment

Instead of $\mathbb{Z}_q$, we can commit to messages in $\mathbb{Z}_q^n$ using random points with unknown discrete logarithms between them.

Definition

The **Pedersen vector commitment scheme** $\Pi_{n\text{-Ped}}$ is defined as follows:

- Setup$(1^\lambda) \to$ pp: agree on a cyclic group $\mathbb{G}$ of order $q = \Theta(2^\lambda)$ and select $g_0, \ldots, g_{n-1}, h \leftarrow\!\!\$\ \mathbb{G}$.
- Com(pp, $\mathbf{m} \in \mathbb{Z}_q^n; r) \to c \in \mathbb{G}$: select $r \leftarrow\!\!\$\ \mathbb{Z}_q$ and output

$$c \leftarrow h^r \prod_{i \in [n]} g_i^{m_i} = g_0^{m_0} g_1^{m_1} \cdots g_{n-1}^{m_{n-1}} h^r.$$

Lemma

$\Pi_{n\text{-Ped}}$ is *perfectly* hiding and *computationally* binding under the discrete logarithm assumption.

The commitment is a **single** group element, regardless of n.

Polynomial Commitments

Polynomial Commitments

So far, our schemes were *commit-reveal*: to convince the verifier, the committer opens the whole message. Often, we want to prove statements about the committed data *without* revealing it.

Definition (informal)

A **polynomial commitment scheme** allows to commit to $f \in \mathbb{F}_q[T]^{<d}$ and later prove that $f(\rho) = y$ for a point ρ chosen by the verifier, without revealing f. The scheme is **succinct** if the commitment and the proof are much smaller than f.

Remark

Such schemes are written in the form “prover sends A, verifier responds with B, . . . ”. Using the Fiat–Shamir transformation, we will make them non-interactive later.

Pedersen Polynomial Commitment

Let $g, h \in \mathbb{G}$ be points with unknown discrete logarithms between them, and $f = \sum_{i \in [d]} f_i T^i$.

Definition

The **Pedersen polynomial commitment scheme** Π_{poly}:

1. Setup$(1^\lambda) \to$ pp: agree on $\mathbb{G}$ of order $q = \Theta(2^\lambda)$ and $g, h \leftarrow\!\$\ \mathbb{G}$.
2. Com(pp, f) $\to ((y, z), \mathbf{c})$: the prover and the verifier interact as follows:
   1: the prover selects $\boldsymbol{\gamma} \leftarrow\!\$\ \mathbb{F}_q^d$ and sends $c_i \leftarrow g^{f_i} h^{\gamma_i}$ for $i \in [d]$;
   2: the verifier samples $\rho \leftarrow\!\$\ \mathbb{F}_q$ and sends it to the prover;
   3: the prover sets $\gamma(T) = \sum_{i \in [d]} \gamma_i T^i$ and outputs $(y, z) \leftarrow (f(\rho), \gamma(\rho))$.
3. Open(pp, $\mathbf{c}, \rho, y, z$): the verifier checks that

$$\prod_{i \in [d]} c_i^{\rho^i} = g^y h^z.$$

Correctness: $\prod_i c_i^{\rho^i} = g^{\sum_i f_i \rho^i} h^{\sum_i \gamma_i \rho^i} = g^{f(\rho)} h^{\gamma(\rho)}$.

Pedersen Polynomial Commitment: Security

Lemma

Π_{poly} is perfectly hiding and computationally binding under the discrete logarithm assumption.

Perfect hiding. Given (ρ, y), one can sample $(\mathbf{c}, z)$ with exactly the same distribution, without knowing f: take random z and all c_i except one, and fix the last c_j so that the check passes.

Computational binding. Two different openings $(y, z) \neq (y', z')$ give

$$\prod_{i \in [d]} c_i^{\rho^i} = g^y h^z = g^{y'} h^{z'},$$

so $(y - y')(z' - z)^{-1}$ is the discrete logarithm of h.

Is This Good Enough?

Π_{poly} is just d independent Pedersen commitments, one per coefficient:

$$\mathbf{c} = (g^{f_0} h^{\gamma_0},\ g^{f_1} h^{\gamma_1},\ \ldots,\ g^{f_{d-1}} h^{\gamma_{d-1}}) \in \mathbb{G}^d.$$

- **Not succinct:** the commitment consists of d group elements.
- **Slow verification:** the verifier computes $\prod_{i \in [d]} c_i^{\rho^i}$, i.e., $O(d)$ exponentiations.

In zero-knowledge proofs, d is the size of the whole computation (millions), so both are unacceptable.

Question

Can we commit to the whole polynomial with a **single** group element, and verify an evaluation in **constant** time?

KZG Commitments

KZG: Basic Idea

Suppose the prover wants to convince the verifier that $f(\rho) = y$ for a random point $\rho \leftarrow\!\!\$\ \mathbb{F}_q$ chosen by the verifier.

Notice that ρ is a root of $f(T) - y$. Hence, there exists a **quotient polynomial** $u \in \mathbb{F}_q[T]^{<d}$ with

$$f(T) - y = (T - \rho)\, u(T).$$

1: Before receiving ρ, the prover commits to f, obtaining $\mathrm{Com}(f)$.
2: After obtaining ρ, the prover commits to u, obtaining $\mathrm{Com}(u)$.
3: The verifier checks the identity $f(T) - y = (T - \rho)u(T)$ at a random point τ (Schwartz–Zippel), using only $\mathrm{Com}(f)$ and $\mathrm{Com}(u)$.

KZG: Toy Example

Given the polynomial $f(T) = T^3 - 15T^2 + 71T - 103$, prove that $f(3) = 2$.

$f(3) = 2 \implies 3$ is a root of $f(T) - 2$.

Proof: the quotient polynomial

$$u(T) = \frac{f(T) - 2}{T - 3} = \frac{T^3 - 15T^2 + 71T - 105}{T - 3} = T^2 - 12T + 35.$$

Verify: $u(T) \cdot (T - 3) = f(T) - 2$.

Question

How can the verifier check this identity at τ without seeing f and u?

KZG: Committing to Polynomials

Let $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ be a pairing, and g_1, g_2 generators of $\mathbb{G}_1, \mathbb{G}_2$. Commit as

$$\mathsf{Com}(f) \leftarrow g_1^{f(\tau)}.$$

Then the identity at τ can be checked with the pairing:

$$e(\mathsf{Com}(f)/g_1^{y}, g_2) = e(\mathsf{Com}(u), g_2^{\tau-\rho}).$$

Problem: the commitment uses τ *before* the challenge, and whoever knows τ can break binding.

Solution: the prover does not need τ itself:

$$g_1^{f(\tau)} = g_1^{\sum_{i\in[d]} f_i \tau^i} = \prod_{i\in[d]} \left(g_1^{\tau^i}\right)^{f_i}.$$

It suffices to publish $\mathsf{pp} := (g_1^{\tau^i})_{i\in[d]}$, which does not reveal τ under the DL assumption.

KZG: Trusted Setup

"Powers-of-tau" ceremony. A trusted party:

1: samples a *toxic parameter* $\tau \leftarrow\$ \mathbb{Z}_q$;

2: publishes $\mathsf{pp} \leftarrow (\{g_1^{\tau^i}\}_{i\in[d]},\ g_2^{\tau})$;

3: **deletes** τ.

Toxic Waste

If τ is revealed, the scheme is broken: anyone can produce a valid proof for *any* value y. This is arguably the largest concern with KZG in practice.

τ-ceremony

Using a distributed generation process, pp can be generated safely given **at least one** honest participant.

KZG Commitment Scheme

Definition

The **KZG commitment scheme** Π_{KZG}:

- $\mathsf{Setup}(1^\lambda) \to \mathsf{pp}$: agree on $\mathbb{G}_1, \mathbb{G}_2$ of order $q = \Theta(2^\lambda)$ with a pairing e; the trusted party outputs $\mathsf{pp} \leftarrow (\{g_1^{\tau^i}\}_{i\in[d]}, g_2^\tau)$ and deletes τ.
- $\mathsf{Com}(\mathsf{pp}, f \in \mathbb{F}_q[T]^{<d}) \to \mathbb{G}_1$: parse $\{h_i\}_{i\in[d]} \leftarrow \mathsf{pp}$ and output $\prod_{i\in[d]} h_i^{f_i} = g_1^{f(\tau)}$.

Definition (KZG Evaluation Proof)

Given $\mathsf{Com}(f)$ sent earlier, the prover shows $f(\rho) = y$ for a random $\rho \leftarrow\!\!\$\ \mathbb{F}_q$:

- $\mathsf{Prove}(\mathsf{pp}, f, \rho, y) \to \pi$: compute $u(T) \leftarrow (f(T) - y)/(T - \rho)$ and output $\pi \leftarrow \mathsf{Com}(\mathsf{pp}, u)$.
- $\mathsf{Verify}(\mathsf{pp}, \mathsf{Com}(f), \rho, y, \pi)$: check

$$e(\mathsf{Com}(f)/g_1^y, g_2) = e(\pi, g_2^\tau/g_2^\rho).$$

KZG: Toy Example (continued)

Back to $f(T) = T^3 - 15T^2 + 71T - 103$, $f(3) = 2$, $u(T) = T^2 - 12T + 35$. Now we run KZG with $q = 17$, where $f(T) = T^3 + 2T^2 + 3T + 16$ and $u(T) = T^2 + 5T + 1$. Group elements are written as g_1^x, g_2^x, so all arithmetic is in the exponent modulo 17.

1: **Setup** with $\tau = 2$: $\quad \mathsf{pp} = (g_1,\ g_1^2,\ g_1^4,\ g_1^8;\ g_2^2)$.

2: **Commit:** $\quad \mathsf{Com}(f) = (g_1)^{16} \cdot (g_1^2)^3 \cdot (g_1^4)^2 \cdot (g_1^8)^1 = g_1^{16+6+8+8} = g_1^{38} = g_1^4$.

3: **Prove:** $\quad \pi = \mathsf{Com}(u) = (g_1)^1 \cdot (g_1^2)^5 \cdot (g_1^4)^1 = g_1^{15}$.

4: **Verify:**

$$e(\mathsf{Com}(f)/g_1^y, g_2) = e(g_1^4/g_1^2, g_2) = e(g_1, g_2)^2,$$
$$e(\pi, g_2^\tau/g_2^\rho) = e(g_1^{15}, g_2^2/g_2^3) = e(g_1, g_2)^{-15} = e(g_1, g_2)^2. \quad 3$$

Why τ must be deleted

Knowing $\tau = 2$, one "proves" $f(3) = 5$ with $\pi' = g_1^{(f(\tau)-5)/(\tau-3)} = g_1^1$: both sides equal $e(g_1, g_2)^{-1}$.

KZG: Correctness and Efficiency

Correctness. By bilinearity, for an honest prover:

$$e(\mathsf{Com}(f)/g_1^y, g_2) = e(g_1, g_2)^{f(\tau)-y},$$
$$e(\pi, g_2^\tau/g_2^\rho) = e(g_1, g_2)^{u(\tau)(\tau-\rho)},$$

and $f(\tau) - y = (\tau - \rho)u(\tau)$ since $f(T) - y = (T - \rho)u(T)$.

	Π_{poly} (Pedersen)	Π_{KZG}
Commitment size	d elements of $\mathbb{G}$	**1** element of $\mathbb{G}_1$
Proof size	2 scalars	**1** element of $\mathbb{G}_1$
Verification	$O(d)$ exponentiations	**2** pairings
Setup	transparent	trusted

KZG: Security

Lemma (informal)

Π_{KZG} is computationally binding: given $\mathsf{Com}(f)$, the prover cannot prove two different values $y \neq y'$ at the same point ρ, under the d-strong Diffie–Hellman (d-SDH) assumption.

d-SDH: given $\{g_1^{\tau^i}\}_{i \leq d}$ and g_2^{τ}, it is hard to output $(c, g_1^{1/(\tau+c)})$ for any c.

Proof sketch. Two accepted proofs π, π' for $y \neq y'$ give

$$e(g_1, g_2)^{y'-y} = e(\pi/\pi', g_2)^{\tau-\rho} \quad \Longrightarrow \quad (\pi/\pi')^{1/(y'-y)} = g_1^{1/(\tau-\rho)},$$

which breaks d-SDH with $c = -\rho$.

KZG is not hiding

$\mathsf{Com}(f) = g_1^{f(\tau)}$ is deterministic. To make it hiding, add a blinding factor $r \leftarrow\!\$\ \mathbb{Z}_q$ and a point $h \in \mathbb{G}_1$:

$$\mathsf{Com}(\mathsf{pp}, f; r) = h^r \prod_{i \in [d]} (g_1^{\tau^i})^{f_i}.$$

KZG: Aggregation for Many Polynomials

The prover committed to $f_0, \ldots, f_{k-1}$ and claims $f_j(\rho) = y_j$ for all $j \in [k]$. Sending k proofs is wasteful.

Homomorphism: $\mathrm{Com}(f) \cdot \mathrm{Com}(f') = g_1^{f(\tau)+f'(\tau)} = \mathrm{Com}(f + f')$.

1: The verifier sends a random $\gamma \leftarrow\!\$\ \mathbb{F}_q$.

2: Both parties combine the claims:

$$h(T) := \sum_{j\in[k]} \gamma^j f_j(T), \quad \mathrm{Com}(h) = \prod_{j\in[k]} \mathrm{Com}(f_j)^{\gamma^j}, \quad y_h := \sum_{j\in[k]} \gamma^j y_j.$$

3: The prover sends **one** proof $\pi \leftarrow \mathrm{Com}\big((h(T) - y_h)/(T - \rho)\big)$, verified as before.

If some claim is false, then $h(\rho) \neq y_h$ except with probability at most $(k-1)/q$ over γ (Schwartz–Zippel).

KZG: Aggregation for Many Points

The prover claims $f(\rho_i) = y_i$ for all $i \in [k]$.

Let $I(T)$ be the interpolating polynomial with $I(\rho_i) = y_i$ ($\deg I < k$) and $Z(T) := \prod_{i \in [k]} (T - \rho_i)$. Then

$$f(\rho_i) = y_i \text{ for all } i \in [k] \iff Z(T) \mid f(T) - I(T).$$

- Prove: compute $u(T) \leftarrow (f(T) - I(T))/Z(T)$ and output $\pi \leftarrow \mathsf{Com}(\mathsf{pp}, u)$.
- Verify: compute $g_1^{I(\tau)}$ and $g_2^{Z(\tau)}$ from pp and check

$$e(\mathsf{Com}(f)/g_1^{I(\tau)}, g_2) = e(\pi, g_2^{Z(\tau)}).$$

The proof is still a **single** element of $\mathbb{G}_1$, regardless of k. The setup must additionally contain $g_2^{\tau^i}$ for $i \leq k$.

Thank you for your attention



research.blockstream.com

github.com/BlockstreamResearch/zk-book



Blockstream